离散数学笔记

上海交通大学, 曹钦翔教授

最后一次更新于 **2026 年 3 月 30 日**

目录

第一章 图论 **1**

1.1 基础概念 1

1.2 连通性 3

1.3 树 5

1.4 图的遍历 9

第二章 逻辑学 **12**

2.1 逻辑连接词和量词 12

2.2 推理 13

第一章　图论

1.1　基础概念

定义 1.1 (图)**.** **图**指的是一个有序对 (V, E), 其中 V 是一个顶点集合, E 是一个边集合. 某条边连接的两个顶点被称为**端点**.

图是对连接性和交叉信息的抽象. 需要注意的是, 边的端点不必是不同的. 换句话说, 允许存在连接某个顶点到它自身的边; 这样的边被称为**自环**.

根据顶点和边的性质, 我们可以将图分为不同的类别. 例如, 根据边是否有方向, 是否存在连接同一对顶点的多条边, 或者是否存在自环等. 特别地, 我们有以下简单图的定义.

定义 1.2 (简单图)**.** 如果一个图是无向的, 且没有多重边或自环, 则称其为**简单图**.

有时候我们也会使用**简单有向图**这个术语, 它指的是满足简单图要求的图, 只是没有无向的限制.

下面引入顶点的度这一概念.

定义 1.3. 假设 $G = (V, E)$ 是一个无向图, 如果存在一条边 $e \in E$ 连接 u 和 v, 则称 u, v **邻接** (或互为**邻接点**), 称这样的边 e **关联**于 u, v.

某个顶点 v 的**邻域** $\mathcal{N}(v)$ 是指 v 的所有邻接点的集合. 如果 $A \subseteq V$, 则 $\mathcal{N}(A)$ 定义为 $\mathcal{N}(A) = \bigcup_{v \in A} \mathcal{N}(v)$.

定义 1.4 (度)**.** 在一个无向图 $G = (V, E)$ 中, 顶点 v 的**度**是指与 v 相关联的边的数量. 注意, 如果存在一条连接 v 到它自身的边 (即一个自环), 则该边会被计算两次. 顶点 v 的度用 $\deg(v)$ 表示.

注意到我们还有一种计算 v 的度的方法:

$$\deg(v) = |\{e \mid v \text{ 是第一端点}\}| + |\{e \mid v \text{ 是第二端点}\}|.$$

因此我们立即得到以下定理, 也被称为*握手定理*.

定理 1.5. *如果 $G = (V, E)$ 是无向图, 则* $\sum_{v \in V} \deg(v) = 2|E|$.

也就是说, 在一个无向图中, 所有顶点的度数之和是边数的两倍.

证明. 因为

$$\deg(v) = \sum_{e \in E} \left(\mathbf{1}_{\{v \text{ 是 } e \text{ 的第一个端点}\}} + \mathbf{1}_{\{v \text{ 是 } e \text{ 的第二个端点}\}} \right),$$

所以

$$\begin{aligned}\sum_{v\in V}\deg(v) &= \sum_{v\in V}\sum_{e\in E}\left(\mathbf{1}_{\{v\text{ 是 }e\text{ 的第一个端点}\}}+\mathbf{1}_{\{v\text{ 是 }e\text{ 的第二个端点}\}}\right)\\ &= \sum_{e\in E}\sum_{v\in V}\left(\mathbf{1}_{\{v\text{ 是 }e\text{ 的第一个端点}\}}+\mathbf{1}_{\{v\text{ 是 }e\text{ 的第二个端点}\}}\right)\\ &= 2|E|.\end{aligned}$$

□

推论 1.6. *在一个无向图中, 度数为奇数的顶点的数量是偶数.*

我们再给出一些在有向图中与无向图不同的术语.

定义 1.7 (有向边)**.** 假设 $G=(V,E)$ 是一个有向图, $e\in E$ 是从 u 到 v 的边, 那么:

- u 是 v 的**前驱**, v 是 u 的**后继**;
- e 的**起点**是 u, e 的**终点**是 v.

在有向图中, 我们还可以定义顶点的**入度**和**出度**.

定义 1.8 (入度和出度)**.** 如果 $G=(V,E)$ 是一个有向图, $v\in V$, 定义

- (入度) $\deg^-(v):=|\{e\in E\mid e\text{ 关联于 }(u',v')\text{ 且 }v=v'\}|$;
- (出度) $\deg^+(v):=|\{e\in E\mid e\text{ 关联于 }(u',v')\text{ 且 }v=u'\}|$.

下述定理刻画了有向图中入度和出度之间的关系.

定理 1.9. *如果 $G=(V,E)$ 是一个有限的有向图, 则*

$$\sum_{v\in V}\deg^-(v)=\sum_{v\in V}\deg^+(v)=|E|.$$

定义 1.10 (路径)**.** 设 $G=(V,E)$ 是一个无向图. 从 u 到 v 的长度为 n $(n\geq 0)$ 的**路径**是指一系列边 $e_1,e_2,\ldots,e_n$, 满足存在一系列顶点 $v_0,v_1,\ldots,v_n$, 使得 $v_0=u$, $v_n=v$, 且对于每个 $i=1,2,\ldots,n$, v_{i-1} 和 v_i 是边 e_i 的端点.

如果一条路径不包含重复的边, 则称其为**简单路径**. 我们称路径 $e_1,\ldots,e_n$ **遍历**这些边, **经过**顶点 $v_0,v_1,\ldots,v_n$.

为简洁起见, 在简单图中, 我们也可以将路径表示为其经过顶点的序列 $v_0,v_1,\ldots,v_n$.

定义 1.11 (回路)**.** **回路**是指起点和终点是同一个顶点, 且长度大于 0 的路径. 如果一条回路不包含重复的边, 则称其为**简单回路**.

值得注意的是, 简单路径经过的顶点不一定互异. 回顾自环的定义, 可以看出**自环是一条长度为 1 的路径, 并且是一条回路.**

在有向图中, 我们也可以以类似的方式定义路径和回路, 只需要额外要求对于每个 $i=1,2,\ldots,n$, e_i 是从 v_{i-1} 到 v_i 的有向边.

1.2 连通性

这一节讨论连通性. 在有向图和无向图中, 连通性的表现是完全不同的, 我们先从较简单的无向图开始讨论.

定义 1.12. 设 $G=(V,E)$ 是一个无向图. 如果存在一条从 u 到 v 的路径, 则称 u 和 v **连通**.

注意: 在这个定义中, 我们没有明确要求路径必须是简单的. 事实上, 如果存在一条从 u 到 v 的路径, 那么一定也存在一条从 u 到 v 的简单路径. 我们在这里不对其加以证明, 但在未来的讨论中我们会使用这一事实.

以下定理表明连通是一个等价关系.

定理 1.13. *在无向图中, 连通是一个等价关系. 也就是说,*

(a) *(自反性) 任意顶点与其自身连通;*

(b) *(对称性) 如果 u 与 v 连通, 则 v 与 u 连通;*

(c) *(传递性) 如果 u 与 v 连通且 v 与 w 连通, 则 u 与 w 连通.*

证明. 对任意顶点 v 来说, 存在一条从 v 到它自身的长度为 0 的路径, 因此 (a) 成立.

如果存在一条从 u 到 v 的路径, 则我们可以反转这条路径来得到一条从 v 到 u 的路径. 用符号表示, 如果 $e_1,\dots,e_n$ 是从 u 到 v 的路径, 则 $e_n,\dots,e_1$ 是从 v 到 u 的路径. 因此 (b) 成立.

如果存在从 u 到 v 的路径和从 v 到 w 的路径, 我们可以将这两条路径首尾衔接, 从而得到一条从 u 到 w 的路径. 用符号表示: 如果 $e_1,\dots,e_n$ 是从 u 到 v 的路径, $f_1,\dots,f_m$ 是从 v 到 w 的路径, 则 $e_1,\dots,e_n,f_1,\dots,f_m$ 是从 u 到 w 的路径. 于是 (c) 得证. □

在定义了顶点之间的连通性之后, 我们就可以定义整个图的连通性了.

定义 1.14 (连通图)**.** 如果在无向图 $G=(V,E)$ 中, 任意两个顶点都是连通的, 则称 G **连通**. 否则, 称 G **不连通**.

在进一步探究连通图的性质之前, 我们先定义一些概念.

定义 1.15 (子图)**.** 设有一个图 $G=(V,E)$. 如果图 $G'=(V',E')$ 满足 $V'\subseteq V$ 且 $E'\subseteq E$, 则称 G' 是 G 的**子图**. 如果还满足 $G'\neq G$, 则称 G' 是 G 的**真子图**.

定义 1.16 (导出子图)**.** 设有图 $G=(V,E)$ 及其顶点子集 $W\subseteq V$. 由 W **导出**的子图是指包含 W 中所有顶点以及 E 中所有连接 W 中顶点的边的子图.

定义 1.17 (连通块)**.** 图 $G=(V,E)$ 中的**连通块**是指 G 的一个最大连通子图. 这里, "最大" 的意思是该子图不是 G 的任何其他连通子图的真子图.

现在来讨论一个关于连通块的重要定理.

定理 1.18. *设 G 是一个非空的无向图, G' 是 G 的一个子图. 那么 G' 是 G 的一个连通块, 当且仅当存在一个顶点 v, 使得 G' 是由与 v 连通的顶点集合 $W=\{u\mid u\text{ 与 }v\text{ 连通}\}$ 导出的子图.*

证明. 设 $G'=(V',E')$ 是 G 的一个连通块, 我们可以从 G' 中任意选择一个顶点 v, 并以此构造集合 W. 首先我们证明 $V'=W$.

如果 $v'\in V'$ 但 $v'\notin W$, 则 v' 与 v 不连通, 这与 G' 是连通的矛盾. 因此 $V'\subseteq W$. 如果 $u\in W$ 但 $u\notin V'$, 则 u 与 v 连通, 因此存在一条从 u 到 v 的路径. 将这条路径添加到 G' 中, 我们可以得到一个包含 G' 的图, 且该图是 G 的连通真子图, 这与 G' 的最大性矛盾. 因此 $W\subseteq V'$. 综上所述, $V'=W$.

然后再证明 G' 是由 W 导出的子图. 如果存在一条边 $e \in E$ 的两个端点都在 W 中, 但 $e \notin E'$, 则我们可以将 e 添加到 G' 中, 从而得到一个包含 G' 的图, 且该图是 G 的连通真子图, 矛盾! 因此 G' 包含 E 中所有连接 W 中顶点的边, 即 G' 是由 W 导出的子图.

反过来, 假设 G' 是由 W 导出的子图. 那么由于 W 中的任意两个顶点在 G 中都是连通的, 根据传递性, G' 在 G 中也是连通的. 并且, G' 包含了 G 中连接 W 中顶点的所有边, 因此这些顶点在 G' 中也是连通的. 用符号表示, 如果 u_1 和 u_2 都在 G' 中, 则 G 中存在一条从 u_1 到 u_2 的路径 $e_1, \cdots, e_n$; 由于这条路径中的顶点都在 W 中, 那么所有的边也必定都在 G' 中. 因此在 G' 中也存在一条从 u_1 到 u_2 的路径.

另一方面, 如果存在一个连通子图 H 使得 G' 为其真子图, 则 H 中存在一个顶点 (显然, 此顶点与 v 连通) 不在 G' 中, 这与 W 的定义矛盾. 因此 G' 是 G 的一个连通块. □

现在我们来考察有向图. 在有向图中, 连通性更为复杂, 不过我们仍然可以定义一种类似于无向图中连通的关系.

定义 1.19 (强连通)**.** 设 $G = (V, E)$ 是一个有向图, $u, v \in V$. 如果在 G 中既存在从 u 到 v 的路径, 又存在从 v 到 u 的路径, 则称 u 和 v **强连通**.

在有向图中, 如果存在一条从 $u \in V$ 到 $v \in V$ 的路径, 我们不称这两顶点连通, 而是说 v 从 u **可达**; 这里要注意两个顶点的顺序. 强连通的顶点也被称为**相互可达**的顶点. 我们也可以以类似于无向图的方式定义**强连通图**和**强连通块**, 这里不再赘述.

显见, 强连通也是等价关系. 证明与无向图中的连通性类似, 这里就不详细展开了.

定理 1.18 也适用于有向图中的强连通块, 只是需要对定理的表述进行一些修改. 下面我们给出定理的有向版本, 略去证明.

定理 1.20. *设 G 是一个非空的有向图, G' 是 G 的一个子图. 那么 G' 是 G 的一个强连通块, 当且仅当存在一个顶点 v, 使得 G' 是由与 v 强连通的顶点集合 $W = \{u \mid u \text{ 与 } v \text{ 强连通}\}$ 导出的子图.*

基本上, 我们只是将定理 1.18 中的 "连通" 替换成了 "强连通".

下面这个定理刻画了强连通块之间连通性的结构.

定理 1.21. *设 G 是一个有向图, $G' = (V', E')$ 和 $G'' = (V'', E'')$ 是 G 的两个不同的强连通块, 则对于任意 $u_1, u_2 \in V'$ 和任意 $v_1, v_2 \in V''$, v_1 从 u_1 可达当且仅当 v_2 从 u_2 可达.*

此定理表明, 如果 G' 中的某个顶点可以到达 G'' 中的某个顶点, 则 G' 中的任意顶点都可以到达 G'' 中的任意顶点.

证明. 如果 v_1 从 u_1 可达, 则存在一条从 u_1 到 v_1 的路径. 由于 u_2 与 u_1 强连通, v_2 与 v_1 强连通, 存在一条从 u_2 到 u_1 的路径和一条从 v_1 到 v_2 的路径. 将这些路径首尾衔接, 就得到了从 u_2 到 v_2 的路径.

另一个方向的证明类似. □

定理 1.22. *如果 G 是一个有向图, $e_1, \ldots, e_n$ 是一条经过 $x_0, x_1, \ldots, x_n = x_0$ 的回路, 则 $x_0, x_1, \ldots, x_n$ 属于 G 的同一个强连通块.*

也就是说, 不存在经过两个不同强连通块的回路.

证明. 用反证法. 假设 $x_0, x_1, \cdots, x_n$ 不属于同一个强连通块, 则存在两个顶点在这个回路中不强连通. 但是, 由于路径 $x_0, \ldots, x_n$ 是一个回路, 对于任意两个顶点 (x_i, x_j), 都存在一条从 x_i 到 x_j 的路径. 交换 x_i 和 x_j 的地位, 就得到了从 x_j 到 x_i 的路径, 与 x_i 和 x_j 不强连通的假设矛盾. □

现在我们来考虑在图中添加或删除边产生的影响, 相关定理如下.

定理 1.23. *设 $G = (V, E)$, $G' = (V, E \cup \{e_0\})$, 其中 $e_0 \notin E$, 且 e_0 连接顶点 u, v. 则*

(a) *对任意 $x, y \in V$, 如果 x 和 y 在 G 中连通, 则它们在 G' 中也连通.*

(b) *如果 u 和 v 在 G 中连通, 则对任意 $x, y \in V$, x 和 y 在 G 中连通当且仅当它们在 G' 中连通.*

(c) *如果 u 和 v 在 G 中不连通, 则对任意 $x \in V$, x 在 G' 中与 u 连通当且仅当 x 在 G' 中与 v 连通; 当且仅当 x 在 G 中与 u 或者与 v 连通.*

(d) *对任意 $x, y \in V$, 如果 x 和 y 在 G 中不连通但在 G' 中连通, 则 x, y, u, v 在 G' 中连通.*

证明. (a) 是显然的, 因为 G 是 G' 的一个子图.

(b) 的其中一个方向是 (a) 的直接推论. 对于另一个方向, 假设 x 和 y 在 G' 中连通, 则 G' 中存在一条从 x 到 y 的路径. 如果这条路径不包含边 e_0, 则它也是 G 中的一条路径, 因此 x 和 y 在 G 中连通. 否则, 这条路径必定包含了边 e_0, 不妨设它是 $x \to u \to v \to y$, 由于 u 和 v 在 G 中连通, 我们可以用 G 中的一条从 u 到 v 的路径替换掉边 e_0, 从而得到一条 G 中的从 x 到 y 的路径. 因此, x 和 y 在 G 中连通.

(c) 的命题有四个不同的方向, 其中三个是显然的. 唯一困难的是证明如果 x 在 G' 中与 u 连通, 则 x 在 G 中与 u 或者与 v 连通.

根据假设, G' 中存在一条从 x 到 u 的路径. 如果这条路径不包含边 e_0, 则它也是 G 中的一条路径, 命题立即得证. 否则, 假设这条路径是 $e_1, \ldots, e_n$, 经过 $x_0 = x, x_1, \ldots, x_m = u$, 且 e_j 是这条路径中第一次出现边 e_0 的位置. 那么 x_{j-1} 与 x 连通, 且 x_{j-1} 是 e_0 的一个端点, 因此 x_{j-1} 要么是 u, 要么是 v. 于是 x 与 u 或者与 v 连通.

(d) 由以下事实得出: G' 中从 x 到 y 的路径必须包含边 e_0. □

1.3 树

树是一类特殊的图.

定义 1.24 (树)**.** **树**指的是不包含任何简单回路的连通无向图.

在 1.2 节中我们曾不加证明地陈述了以下事实, 现在给出证明.

引理 1.25. *如果存在一条从 u 到 v 的路径, 则一定也存在一条从 u 到 v 的简单路径.*

证明. 由于路径的长度有下界, 在所有从 u 到 v 的路径中, 必定存在一条最短的路径. 如果这条路径不是简单路径, 假定它是 $e_1, \ldots, e_n$, 经过 $x_0 = u, x_1, \ldots, x_n = v$, 且满足 $e_i = e_j = e_0$ $(i \neq j)$. 那么我们可以将 e_0 从路径中删去. 这里分为两种情况: $x_{i-1} = x_{j-1}$, 或者 $x_{i-1} = x_j$. 在第一种情况下, 我们直接将 x_{i-1} 和 x_j 连接起来; 在第二种情况下, 我们将 x_{i-1} 和 x_{j+1} 连接起来. 因此在两种情况下, 我们都可以构造一条从 u 到 v 的更短路径, 矛盾! □

下述定理刻画了树的一个重要性质, 即树是没有回路的连通图中最大的图, 也是连通图中最小的图.

定理 1.26. *向树中添加一条边, 就会产生一个简单回路; 从树中删除一条边, 就会使其不连通.*

证明. 如果在 u, v 之间添加一条边, 由于 u 和 v 已经连通, 我们可以构造一条路径 $u, \ldots, v, u$, 显然这是一个回路.

如果删去 u, v 之间的边后, 图仍然保持连通, 那么根据此证明的第一部分, 将这条边添加回去会在树中产生一个简单回路, 矛盾! □

根据是否指定一个顶点为根, 树可以分为**有根树**和**无根树**. 我们先讨论无根树.

定义 1.27 (叶子)**.** 在树中, 如果一个顶点的度数为 1, 则称其为**叶子**.

以下定理保证了树中存在叶子.

定理 1.28. *如果一棵树至少有两个顶点, 则它至少有一个叶子.*

证明. 由于树是连通的, 我们可以任选一个顶点 v, 并找到一条从 v 到某个顶点 u 的最长路径. 那么 u 是一个叶子, 否则我们可以将路径延长, 从而得到一条更长的路径, 产生矛盾. □

推论 1.29. *如果一棵树至少有两个顶点, 则它至少有两个叶子.*

证明. 在定理 1.28 的证明中我们得到了一个叶子 u. 现在我们再次以 u 作为起点进行同样的操作来找到另一个叶子. 显然新找到的叶子不可能是 u, 因为树中不存在回路. 因此我们找到了两个不同的叶子. □

下面这个定理从顶点数和边数的角度刻画了树的性质.

定理 1.30. *如果 $G=(V,E)$ 是一棵树, 则 $|E|=|V|-1$.*

证明. 对顶点的个数用数学归纳法.

若 $|V|=1$, 则 $|E|=0$, 命题成立.

设命题对于任意顶点数小于 n 的树都成立, 其中 $n\geq 2$. 现在我们考虑一棵有 n 个顶点的树 G. 根据定理 1.28, G 有一个叶子 v. 设 u 是 v 的唯一邻接点. 我们可以删去 u 和 v 之间的边以及顶点 v, 得到一个新的图 $G'=(V',E')$. 显见 G' 仍然是一棵树, 且 $|V'|=n-1$. 根据归纳假设, $|E'|=|V'|-1=n-2$. 由于从 G 到 G' 删去了一条边, 即有 $|E|=|E'|+1=n-1=|V|-1$. □

事实上, 定理 1.26 的逆命题也成立, 这为我们提供了另一种刻画树的手段.

定理 1.31. *在一个不含简单回路的无向图中, 如果添加任意一条边都会在新图中产生一个简单回路, 则该图是树; 在一个连通的无向图中, 如果删除任意一条边都会使新图不连通, 则该图是树.*

证明. 设 $G=(V,E)$ 是一个不含简单回路的无向图, 且对任意 $u,v\in V$, 如果在 G 中添加一条连接 u 和 v 的边, 不妨记作 e_0, 则在新图中存在一条简单回路. 现欲证明 u 和 v 在 G 中连通. 由于 G 中没有简单回路, 新图中的简单回路必须包含边 e_0. 从此回路中删除边 e_0, 就得到了 G 中的一条从 u 到 v 的路径. 因此 G 是连通的, 从而是树.

接下来, 假设 $G=(V,E)$ 是一个连通的无向图, 且如果我们删除 G 中的任意一条边, 新图就会变得不连通. 为了证明 G 是树, 只需证明 G 不包含任何简单回路. 若不然, 假设 $e_1,\ldots,e_n$ 是 G 中的一条简单回路, 且经过顶点 $v_0,v_1,\ldots,v_n=v_0$. 那么我们可以删除边 e_1, 得到一个新图. 由于在新图中仍然存在从 v_0 到 v_1 的路径, 因此新图仍然是连通的, 这与假设矛盾. □

简而言之, 对无向图 $G=(V,E)$ 而言, 以下陈述是等价的:

(a) G 是树, 即 G 是不包含任何简单回路的连通图.

(b) G 是连通的, 且 $|E|=|V|-1$.

(c) G 不包含任何简单回路, 且 $|E|=|V|-1$.

(d) G 是连通的, 但 E 中的每条边都是割边.

(e) G 不包含任何简单回路, 但向 G 中添加任何一条边都会产生一个简单回路.

(f) G 中任意两顶点之间存在唯一的简单路径.

现在来讨论一种特殊的树, 称为生成树.

定义 1.32 (生成树)**.** 在无向图 $G=(V,E)$ 中, 如果 $T=(V,E')$ 是 G 的一个子图, 且 T 是树, 则称 T 是 G 的一棵**生成树**.

定理 1.33. *设 G 是无向图, 则 G 有生成树当且仅当 G 是连通的.*

证明. 若 G 有生成树, 那么由于生成树是连通的, G 也是连通的.

反过来, 若 G 是连通的, 则我们可以从一个空图开始, 逐条添加边来构造一棵生成树. 当且仅当添加一条边不会在新图中产生简单回路时, 我们才添加这条边. 由于 G 是连通的, 可以一直添加边直到所有顶点都连通. 最终得到的图就是 G 的一棵生成树. □

在实际应用中, 我们通常更关心具有某种最优性的生成树, 比如所谓 "最短" 的生成树. 为此, 引入带权图的概念, 以及在带权图中定义的最小生成树.

定义 1.34 (无向带权图)**.** 在无向图 $G = (V, E)$ 中加入权值函数 $w\colon E \to \mathbb{R}$ 后, G 就成为一个**无向带权图**. 权重函数为每条边赋予了一个实数权值.

定义 1.35 (最小生成树)**.** 设 $G = (V, E)$ 是一个连通的带权图. 如果 T 是 G 的一棵生成树, 且 T 中所有边的权值之和在 G 的所有生成树中是最小的, 则称 T 是 G 的**最小生成树**. 用符号来表示, 就是: 若 $T' = (V, E')$ 是 G 的任意一棵生成树, 则

$$\sum_{e \in E} w(e) \leq \sum_{e \in E'} w(e).$$

Prim 算法和 Kruskal 算法是两种经典的求解最小生成树问题的算法, 下面分别介绍.

算法 1.36 (Prim 算法)**.** 一个图 $G = (V, E)$ 的最小生成树可以通过以下步骤构造出来:

(a) 从 G 中任意选择一个顶点, 构造初始树 $T_0 = (V_0, E_0)$, 其中 V_0 仅包含此顶点, E_0 是空集.

(b) 向 T_n 中添加一条边 e 来构造 T_{n+1}, 其中 e 是所有剩余边 e' 中权值最小的边, 且这些 e' 满足 T_n 加上 e' 后仍然是一棵树.

(c) 最终得到的图 $T_{|V|-1}$ 就是 G 的一棵最小生成树.

证明. 首先证明 $T_{|V|-1}$ 是 G 的一棵生成树. 由于我们逐条添加边, $T_{|V|-1}$ 必定是一棵树. 此外, 由于我们一直添加边直到所有顶点都连通, $T_{|V|-1}$ 包含了 V 中的所有顶点. 因此 $T_{|V|-1}$ 是 G 的一棵生成树.

现在先证明一个结论: 对于任意 $0 \leq n \leq |V| - 1$, 存在 G 的一棵最小生成树 T^*, 使得 T_n 是 T^* 的一个子图. 我们对 n 用数学归纳法.

若 $n = 0$, 则 T_0 是一个单独的顶点, 显然是 G 的任意一棵生成树的子图.

现假定命题对 $n \geq 0$ 成立. 对于 $n + 1$, 由归纳假设, 存在 G 的一棵最小生成树 T^*, 使得 T_n 是 T^* 的一个子图. 如果 e 也是 T^* 中的一条边, 则 T_{n+1} 立即是 T^* 的一个子图, 命题得证. 否则, 由于 T^* 是一棵生成树, 其中必定存在一条连接 e 的两个端点的路径. 这条路径必须包含一条不在 T_n 中的边 e' (否则将 e 添加到 T_n 中会产生一个简单回路). 现在用 e 替换 e', 得到一棵新的生成树. 由于 Prim 算法选择添加 e, 这说明 $w(e) \leq w(e')$, 因此这棵新的生成树也是最小生成树, 且 T_{n+1} 为其子图. 因此命题对 $n + 1$ 成立.

最后, 根据 $n = |V| - 1$ 的情况, 存在一棵最小生成树包含 $T_{|V|-1}$ 作为其子图. 由于它们都是生成树, 这两棵树必定相同. 因此 $T_{|V|-1}$ 是 G 的一棵最小生成树. □

算法 1.37 (Kruskal 算法)**.** 一个图 $G = (V, E)$ 的最小生成树可以通过以下步骤构造出来:

(a) 将 E 中的边按照权值从小到大排序, 记排序后的边为 $e_1, e_2, \ldots, e_m$, 其中 $m = |E|$. 初始时令 $T = \varnothing$.

(b) 对 $i = 1, 2, \ldots, m$, 如果将 e_i 添加到 T 中不会产生简单回路, 则将 e_i 添加到 T 中.

(c) 当 T 中的边数达到 $|V| - 1$ 时停止. 此时 T 就是 G 的一棵最小生成树.

Kruskal 算法的证明与 Prim 算法的证明非常相似, 这里不再展开详细讨论.

先前讨论的都是无根树. 现在我们来讨论有根树, 它的结构更类似于我们在现实生活中看到的树.

定义 1.38 (有根树)**.** 在树中指定一个顶点为**根**, 这棵树就成为**有根树**.

注意: 有根树并不一定是有向图. 在绘制有根树时, 通常将根绘制于树的顶部, 并向下绘制各条边.

定义 1.39. 在以 r 为根的有根树中, 定义

(a) 顶点 v 的**层级**为从 r 到 v 的唯一简单路径的长度;

(b) 树的**高度**为树中所有顶点的层级的最大值.

定义 1.40. 设 $G=(V,E)$ 是以 r 为根的有根树, 则对于 $v\neq r$, v 的**父节点**是唯一的一个顶点 u, 满足 $\{u,v\}\in E$ 且这条边出现在从 r 到 v 的唯一简单路径上. 反过来, v 称为 u 的**子节点**.

有根树中的层次结构与我们在现实生活中看到的家族谱系非常相似, 下面的定义描述了这种相似性.

定义 1.41. 在以 r 为根的有根树 $G=(V,E)$ 中,

(a) 如果两个顶点 u,v 有相同的父节点, 则称它们是**兄弟节点**;

(b) 对于 $v\neq r$, v 的**祖先节点**是指从 r 到 v 的唯一简单路径上的所有顶点, 包括 r, 但是不包括 v; v 的**子孙节点**是指所有以 v 为一个祖先节点的顶点.

定理 1.42. *在以 r 为根的有根树中, 如果 v 是 u 的一个祖先节点, w 是 v 的一个祖先节点, 则 w 也是 u 的一个祖先节点.*

换句话说, 祖先关系具有传递性.

证明. 由于 v 是 u 的一个祖先节点, 从 r 到 u 的唯一简单路径经过 v. 由于 w 是 v 的一个祖先节点, 从 r 到 v 的唯一简单路径经过 w. 将这两条路径相连, 就得到了从 r 到 u 的一条简单路径, 且这条路径经过 w. 因此 w 是 u 的一个祖先节点. □

子孙关系也有类似的传递性 (即子孙的子孙仍然是子孙), 这可以直接从定义中得出.

引理 1.43. *在有根树中, 若 u 是 v 的父节点, 且 $u=x_n,x_{n-1},\ldots,x_0=r$ 是从 u 到根 r 的简单路径, 则 $v,u=x_n,x_{n-1},\ldots,x_0=r$ 也是一条简单路径.*

证明. 设 $v=y_m,y_{m-1},\ldots,y_0=r$ 是从 v 到 r 的简单路径, 则据定义有 $u=y_{m-1}$. 由于树中任意两顶点之间的简单路径是唯一的, 必定有 $n=m-1$, 且 $x_n=y_{m-1}=u$, $x_{n-1}=y_{m-2},\ldots,x_0=y_0=r$. 因此 $v,u=x_n,x_{n-1},\ldots,x_0=r$ 是一条简单路径. □

定理 1.44. *在有根树中, u 是 v 的一个祖先节点当且仅当存在一列顶点 $v=x_n,x_{n-1},\ldots,x_0=u$, 使得对任意 $i=1,2,\ldots,n$, x_{i-1} 是 x_i 的父节点.*

证明. 若 u 是 v 的祖先节点, 设从 v 到根 r 的简单路径为 $v=y_m,y_{m-1},\ldots,y_0=r$, 则存在 k 使得 $y_k=u$. 取 $x_n=v,x_{n-1}=y_{m-1},\ldots,x_0=y_k=u$ 即得所求的顶点列.

反过来, 如果存在一列顶点 $v=x_n,x_{n-1},\ldots,x_0=u$ 使得对任意 $i=1,2,\ldots,n$, x_{i-1} 是 x_i 的父节点, 记从 u 到根 r 的简单路径为 $u=z_k,z_{k-1},\ldots,z_0=r$, 则由引理 1.43 可知 $v,x_{n-1},\ldots,u,z_{k-1},\ldots,z_0=r$ 也是一条简单路径. 因此 u 是 v 的一个祖先节点. □

引理 1.45. *在有根树中, 如果 u 是 v 的一个祖先节点, 则从 u 到 v 的简单路径上的顶点要么是 u, 要么是 u 的子孙节点.*

证明. 设从 v 到根 r 的简单路径为 $v=y_m,y_{m-1},\ldots,y_0=r$. 由于 u 是 v 的一个祖先节点, 存在 k 使得 $y_k=u$. 那么从 u 到 v 的简单路径就是 $u=y_k,y_{k+1},\ldots,y_m=v$. 显然这条路径上除了 u 以外的所有顶点都是 u 的子孙节点. □

定义 1.46 (叶子和内部顶点)**.** 在有根树中, 没有子节点的顶点称为**叶子**; 否则称为**内部顶点**.

定义 1.47 (子树)**.** 一个有根树中, 顶点 v 处的**子树**是指由 v 及其所有子孙节点导出的子图.

注意: 尽管名为 "子树", 但定义中并没有明确规定子树必须是一棵树. 下面我们证明子树确实是树.

定理 1.48. *有根树中任意一个顶点处的子树仍然是一棵树.*

证明. 子树中显然不含有简单回路, 因此我们只需要证明子树是连通的. 那么只需证明子树中的任意顶点都与 v 连通. 这直接由子树的定义即可得到, 因为子树中的每个顶点都是 v 的一个子孙节点. □

定理 1.49. *在有根树中, 如果 v_1 和 v_2 都是 u 的子孙节点, 则从 v_1 到 v_2 的简单路径上的顶点要么是 u, 要么是 u 的子孙节点.*

证明. 由于 v_1 和 v_2 都是 u 的子孙节点, 它们都在 u 处的子树中. 因此从 v_1 到 v_2 的简单路径也在 u 的子树中. 根据定义, 这条路径上的所有顶点都是 u 的子孙节点, 除了 u 自身. □

定义 1.50 (m 叉树)**.** 设 G 是一棵有根树,

(a) 如果 G 中每个内部顶点的子节点数都不超过 m, 就称 G 是 m **叉树**;

(b) 如果 G 是 2 叉树, 就称其为**二叉树**.

定理 1.51. *高度为 h 的 m 叉树至多有 m^h 个叶子.*

证明. 我们对 h 用数学归纳法. 若 $h = 0$, 则树中只有一个顶点, 也是一个叶子, 因此命题成立.

设命题对 $h \geq 0$ 成立, 现在考虑一棵高度为 $h + 1$ 的 m 叉树. 树的根至少有一个子节点, 且每个子节点都是一棵高度不超过 h 的 m 叉树的根. 根据归纳假设, 每个子节点处的子树中至多有 m^h 个叶子. 由于根的子节点数不超过 m, 因此树中总共至多有 $m \cdot m^h = m^{h+1}$ 个叶子. □

1.4 图的遍历

算法 1.52 (深度优先搜索, DFS)**.** 若 G 是一个连通图, 我们可以通过以下步骤遍历 G 中的所有顶点:

(a) 从 G 中任意选择一个顶点 v 作为起点.

(b) 不断地将一条简单路径进行扩展, 直到没有边可以扩展为止.

(c) 回溯到最深的一个顶点, 使得从该顶点出发仍然有边可以扩展. 重复步骤 (b).

定义 1.53. 如果在第 k 步中, 从顶点 u 通过边 e 移动到顶点 v, 且在第 k 步之前没有访问过 v, 则称这是一次**前向移动**; 如果在第 k 步之前访问过 v, 则称这是一次**后向移动**.

更正式地说, 设起点是 v_0, 在第 k 步中, 从 v_{k-1} 通过边 e_k 移动到 v_k.

(a) 如果存在 $e \in E$, $w \in V$ 使得 $w \notin \{v_0, \ldots, v_{k-1}\}$, 且 e 连接 v_{k-1} 和 w, 则第 k 步是前向移动; 否则第 k 步是后向移动.

(b) 如果第 k 步是前向移动, 则 $v_k \notin \{v_0, \ldots, v_{k-1}\}$.

(c) 如果第 k 步是后向移动, 且 $v_{k-1} = v_i$, 其中 $0 \leq i < k - 1$ 满足第 i 步是前向移动, 则 $v_k = v_{i-1}$, $e_k = e_i$.

有时, 我们也会将其简称为**第 k 次前向移动**或者**第 k 次后向移动**.

定义 1.54. 在 DFS 中, 如果存在一次通过边 e 的前向移动, 则称 e 是一条**树边**; 否则, 称 e 是一条**回边**.

定理 1.55. *在连通无向图 G 中, 一次 DFS 中的所有树边构成了 G 的一棵生成树.*

这正是 "树边" 这个名字的由来.

证明. 首先, 我们证明 DFS 中的所有树边构成的子图包含了 G 中的所有顶点. 反证法: 如果顶点 v 没有出现在 DFS 的过程中, 由于 G 是连通的, 存在一条从 v 到某个出现在 DFS 过程中的顶点 u 的路径. 从 u 出发沿着这条路径走下去, 最终会到达 v, 导出矛盾.

然后用数学归纳法证明子图 $G_k = (V_k, E_k)$ 是一棵树, 这里 $V_k = \{v_0, \ldots, v_k\}$, $E_k = \{e_1, \ldots, e_k\}$, 且规定 $E_0 = \varnothing$.

$k = 0$ 的情形是显然的. 现假设命题对 $k \geq 0$ 成立. 如果第 $k+1$ 步是后向移动, 则 $G_{k+1} = G_k$, 因此命题成立. 否则第 $k+1$ 步是前向移动, 此时 G_{k+1} 是通过在 G_k 的基础上添加一个新顶点和一条新的边得到的. 由于 e_{k+1} 连接 G_k 中的一个顶点和 G_k 外的一个顶点, 因此添加 e_{k+1} 不可能在新图中产生简单回路, 故 G_{k+1} 仍然是一棵树. 于是归纳成立, 从而完成证明. □

通常, 我们将生成树视为以起点为根的有根树.

推论 1.56. *一次前向移动是从一个顶点移动到它的一个子节点.*

引理 1.57. *设 $G = (V, E)$ 是连通无向图, $u \in V$. 则在以 u 为起点的 DFS 过程中, 任一顶点 $v \neq u$ 都恰有一次作为前向移动的终止顶点出现.*

引理 1.58. *设 $G = (V, E)$ 是连通无向图, 在 G 上进行的一次 DFS 过程共有 s 步, 其中第 k 步是从 v_{k-1} 通过边 e_k 移动到 v_k. 则对于任意 $k \leq s$, "已访问顶点" $\{v_0, \ldots, v_k\}$ 可以划分为以下几类:*

(a) *起始顶点;*

(b) *在前 k 步中恰有一次作为前向移动的终止顶点, 且不作为任何一次后向移动的起始顶点的顶点;*

(c) *在前 k 步中恰有一次作为前向移动的终止顶点, 且恰有一次作为后向移动的起始顶点的顶点.*

更进一步, 从 v_0 到 v_k 的一条简单路径满足以下性质:

(a) *这条路径仅包含树边 $e_{n_1}, e_{n_2}, \ldots, e_{n_l}$, 其中 $1 \leq n_1 < n_2 < \cdots < n_l \leq k$;*

(b) *这条路径经过顶点 $v_{n_1-1}, v_{n_1}, v_{n_2}, \ldots, v_{n_l}$, 其中 $v_{n_l} = v_k$ 且 $v_{n_{i-1}} = v_{n_i-1}$;*

(c) *集合 $\{v_0, v_{n_1}, \ldots, v_{n_l}\}$ 恰好是上述类别 (a) 和 (b) 中所有顶点的集合.*

证明. 对 k 用数学归纳法. 若 $k = 0$, 命题显然成立. 假设命题对 $k \geq 0$ 成立, 现在考虑 $k+1$ 的情形.

如果第 $k+1$ 步是后向移动, 则对顶点的唯一改变就是此步的起始顶点 v_k 从类别 (b) 变成了类别 (c). 另外, 由于 $v_{k+1} = v_i$, 其中 $i < k$, 从 v_0 到 v_{k+1} 的简单路径就是从 v_0 到 v_i 的简单路径, 根据归纳假设, 此路径满足所有性质.

如果第 $k+1$ 步是前向移动, 则 v_{k+1} 属于类别 (b), 而从 v_0 到 v_{k+1} 的简单路径就是从 v_0 到 v_k 的简单路径加上边 e_{k+1}. 容易验证这条新路径满足所有性质. 证毕. □

推论 1.59. *设 $G = (V, E)$ 是连通无向图, $u \in V$, 则在以 u 为起点的 DFS 过程中, 任一顶点 $v \neq u$ 都恰有一次作为后向移动的起始顶点出现.*

推论 1.60. *设 $G = (V, E)$ 是连通无向图, $|V| = n$, 则在 G 上进行的一次 DFS 过程共包含 $n-1$ 次前向移动和 $n-1$ 次后向移动.*

推论 1.61. *设 $G = (V, E)$ 是连通无向图, 在 G 上进行的一次 DFS 过程共包含 s 步, 其中第 k 步是从 v_{k-1} 通过边 e_k 移动到 v_k. 如果 $v_i = v_j = u$ 且 $i < j$, 则对任意 $i < k < j$, v_k 都是 u 的一个子孙节点.*

定理 1.62. *一条回边要么连接某个顶点及其祖先节点或子孙节点, 要么是一个自环.*

证明. 考察一条连接 u 和 v 的边. 如果 u, v 中有一者是根, 则命题显然成立. 否则, 分别考察离开 u 和 v 的后向移动. 不失一般性, 假设离开 u 的后向移动更早发生, 那么离开 u 的时候, v 必定已经被访问过了 (否则就应该是前向移动而非后向移动). 由于离开 v 的后向移动还没有发生, 因此 v 必定是 u 的一个祖先节点. □

算法 1.63 (Tarjan 求桥算法)**.**

```
procedure TARJAN-BRIDGE(u, e_0)
    time ← time + 1
    dfn[u] ← time
    low[u] ← time
    for each edge e ≠ e_0 connecting u and v do
        if dfn[v] is not initialized then
            TARJAN-BRIDGE(v, e)
            low[u] ← min{low[u], low[v]}
            if low[v] = dfn[v] then
                e is a bridge
            end if
        else
            low[u] ← min{low[u], dfn[v]}
        end if
    end for
end procedure
```

第二章　逻辑学

2.1　逻辑连接词和量词

定义 2.1. 如果 p 和 q 是两个命题, 那么

(a) p 和 q 的**合取**, 记为 $p \wedge q$, 是命题 "p 和 q". $p \wedge q$ 当且仅当 p 和 q 都为真时为真, 否则为假.

(b) p 和 q 的**析取**, 记为 $p \vee q$, 是命题 "p 或 q". $p \vee q$ 当且仅当 p 和 q 至少有一个为真时为真, 否则为假.

(c) p 的**否定**, 记为 $\neg p$, 是命题 "非 p". $\neg p$ 的真值与 p 的真值相反.

例 2.2. 设 $p =$ "现在正在下雨" 和 $q =$ "我有一把伞". 那么 $p \wedge q$ 是命题 "现在正在下雨并且我有一把伞", $p \vee q$ 是命题 "现在正在下雨或者我有一把伞", $\neg p$ 是命题 "现在没有下雨".

定义 2.3. 定义两个量词如下:

(a) **全称量词** $\forall$ 用来表示 "对于所有" 或 "对于每一个".

(b) **存在量词** $\exists$ 用来表示 "存在" 或 "对于某些".

我们可以将量词与逻辑连接词组合起来, 以构造更复杂的命题.

例 2.4. 令 $E(t) =$ "t 是偶数" 和 $P(t) =$ "t 是质数". 那么 $\exists t \quad (E(t) \wedge P(t))$ 是命题 "存在一个 t, 使得 t 是偶数且 t 是质数".

定义 2.5. **条件语句** $p \to q$ 是指命题 "若 p 则 q", 它当且仅当 p 为真且 q 为假时为假, 否则为真.

$p \to q$ 也可以读作 "p 蕴涵 q", 因此也叫做**蕴涵式**. 通常它与全称量词 $\forall$ 一起使用. 例如, $\forall t \quad (S(t) \to C(t))$ 是读作 "对于所有 t, 若 $S(t)$ 则 $C(t)$".

既然 $\forall$ 与 $\to$ 配对, 那么 $\exists$ 与什么配对呢? 事实上, $\exists$ 与 $\wedge$ 配对. 例如, $\exists t \quad (U(t) \wedge M(t))$ 是命题 "存在一个 t, 使得 $U(t)$ 和 $M(t)$ 都为真". 如果我们设 $U(t) =$ "t 是一个美国学生", $M(t) =$ "t 访问过墨西哥", 那么 $\exists t \quad (U(t) \wedge M(t))$ 就是命题 "存在一个访问过墨西哥的美国学生".

现在来考察一些比较复杂的例子.

例 2.6. 自然语言 "SAI 的每个学生都会被分配一位研究导师" 可以翻译成以下命题:

$$\forall x \quad (S(x) \to \exists y \quad M(x, y))$$

其中 $S(t) =$ "t 是 SAI 的学生", $M(t_1, t_2) =$ "t_2 是分配给 t_1 的研究导师".

自然语言 "SAI 的某位学生如果不努力学习, 就会得到一个非常差的分数" 可以翻译成以下命题:

$$\forall x \quad (S(x) \wedge \neg H(x)) \to B(x)$$

其中 $S(t) =$ "t 是 SAI 的学生", $H(t) =$ "t 努力学习", $B(t) =$ "t 得到一个非常差的分数". 注意, 自然语言中的 "某位" 是个比较模糊不清的说法. 这个句子应当理解成 "对于 SAI 的每个学生, 如果他 (她) 不努力学习, 那么他 (她) 就会得到一个非常差的分数". 这就是为什么翻译使用了 $\forall$ 而非 $\exists$.

例 2.7. 现在来想想如何使用量词和逻辑连接词表达经典的数学命题 “存在唯一一个 x, 使得 $P(x)$”. 这个命题可以表述为

$$\exists x \quad (P(x) \wedge \forall y \quad (P(y) \to y = x)).$$

这个命题的意思是存在一个 x, 使得 $P(x)$ 为真, 并且对于所有 y, 如果 $P(y)$ 为真, 那么 y 必须等于 x. 换句话说, 满足性质 P 的 x 是唯一的.

2.2 推理

我们从一个简单的例子开始. 假设有以下命题:

$$p\colon T \text{ 是连通的且没有简单回路.}$$
$$q\colon T \text{ 是连通的.}$$

可以从 p 推出 q. 换句话说, 如果 p 为真, 那么 q 必须为真. 我们可以用符号来表示这个推理: $p \wedge q \vdash q$. 注意, 在本例中, 我们不需要知道连通图或者简单回路的确切定义, 而只需要知道 p 蕴涵 q.

还有一个特殊的例子: $\vdash p \vee \neg p$. 在这个例子中, 不需要任何假设.

下面正式定义符号 $\vdash$.

定义 2.8. 如果 Φ 是一个命题集合, ψ 是一个命题, 我们用 $\Phi \vdash \psi$ 表示 Φ **可以推出** ψ. $\vdash \psi$ 是 $\varnothing \vdash \psi$ 的简写, $\Phi, \phi_1, \ldots, \phi_n \vdash \psi$ 是 $\Phi \cup \{\phi_1, \ldots, \phi_n\} \vdash \psi$ 的简写.